



SDU International Summer School

# Introduction to R

*Day 1 - Fundamentals*

Paola Vazquez-Castillo

03 August 2026

## Contents

|          |                                              |           |
|----------|----------------------------------------------|-----------|
| <b>1</b> | <b>Curriculum</b>                            | <b>2</b>  |
| <b>2</b> | <b>R</b>                                     | <b>2</b>  |
| <b>3</b> | <b>RStudio</b>                               | <b>3</b>  |
| <b>4</b> | <b>Before starting</b>                       | <b>4</b>  |
| 4.1      | Organizing your work . . . . .               | 4         |
| 4.2      | Good practice advices . . . . .              | 4         |
| 4.3      | Using AI tools in this course . . . . .      | 5         |
| <b>5</b> | <b>Create an object</b>                      | <b>5</b>  |
| 5.1      | Some rules for creating an object: . . . . . | 5         |
| <b>6</b> | <b>Data type</b>                             | <b>6</b>  |
| 6.1      | Numeric . . . . .                            | 6         |
| 6.2      | Integer . . . . .                            | 6         |
| 6.3      | Logical . . . . .                            | 7         |
| 6.4      | Character . . . . .                          | 7         |
| 6.5      | Factor . . . . .                             | 7         |
| 6.6      | Convert from one type to another . . . . .   | 8         |
| <b>7</b> | <b>Vector</b>                                | <b>9</b>  |
| 7.1      | Creating vector with : . . . . .             | 9         |
| 7.2      | Creating vector with seq() . . . . .         | 9         |
| 7.3      | Selecting elements of a vector . . . . .     | 10        |
| 7.4      | Modyfing elements of a vector . . . . .      | 10        |
| 7.5      | Name the elements . . . . .                  | 11        |
| <b>8</b> | <b>Basic arithmetic</b>                      | <b>11</b> |

|                                 |           |
|---------------------------------|-----------|
| <b>9 Functions</b>              | <b>12</b> |
| <b>10 Importing a dataset</b>   | <b>13</b> |
| <b>11 Basic plot</b>            | <b>14</b> |
| <b>12 Glossary of functions</b> | <b>15</b> |
| <b>13 Exercises</b>             | <b>16</b> |

## 1 Curriculum

- Day 1: Fundamentals
- Day 2: Data structures
- Day 3: Conditions, loops, and functions
- Day 4: Packages and introduction to data wrangling
- Day 5: Data wrangling with tidyverse
- Day 6: Visualization
- Day 7: Basic statistics
- Day 8: Linear and multiple regression
- Day 9: Workshop
- Day 10: Exam
- Re-exam: September

Evaluation is an on-premises examination of 2 hours. Class notes are allowed in physical and digital form (PDF, Word or R scripts). Access to the internet and to AI tools is not allowed. You hand in both an R script and a document (PDF or Word) with your results and interpretations.

## 2 R

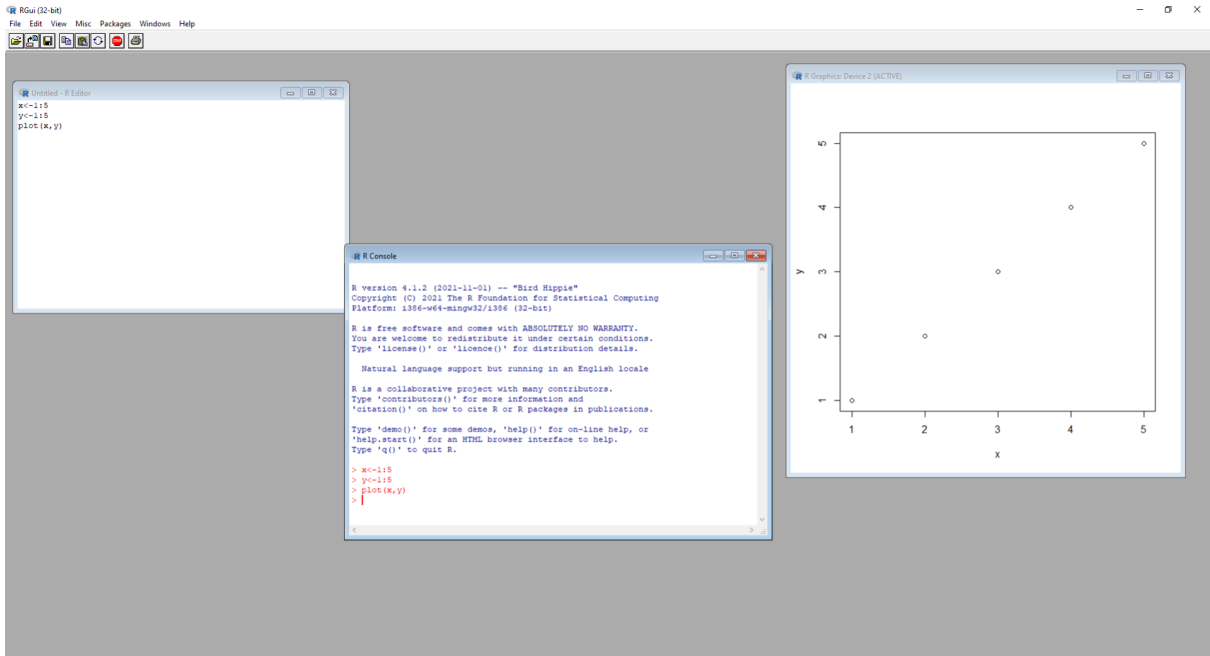
R is a programming language and environment for data manipulation, statistical computing, and data visualization. The developers refer to R as an environment, rather than a statistical software, as it is a fully planned and coherent system. R is also multiplatform, meaning that it is available for Windows, Macintosh and GNU/Linux distributions. Moreover, it is a tool that is compatible with many other programs (e.g. *Latex*) and programming environments (e.g. `sql`, `python`).

Advantages:

- Open source: No license, no fee and everyone can contribute.
- RStudio: R's IDE (integrated development environment) is simple but powerful.
- Reproducibility: Store information in scripts.
- Packages: Great diversity of functions and quick implementation of new approaches.
- Great support for data wrangling (`dplyr`).
- High quality graphics.
- Highly compatible with other languages (e.g. C, C++, Python, etc.).
- Interdisciplinary but strongest suit for statistics and machine learning.
- Continuously growing.
- Supporting community (e.g. cheat Sheets).

Disadvantages:

- Not ideal when dealing with big data set due to the computer's physical memory (alternative: Python).
- Not a general-purpose programming language (alternative: Java).
- Steep learning curve (alternative: Stata).
- Slower than other languages (alternative: MATLAB and Python).



### 3 RStudio

R's graphical user interface (GUI) is not very friendly; thus we interact with R via **RStudio**. **RStudio** is an integrated development environment (IDE) for R. Its interface is organized so you can view the console, script, graphs, history and workspace management at the same time.

If you have not modified the appearance, the **RStudio console** is in the bottom left window. The console is the space where R executes commands and shows the results of the executed code. It is always in standby waiting for instructions. It can also operate as a “calculator”.

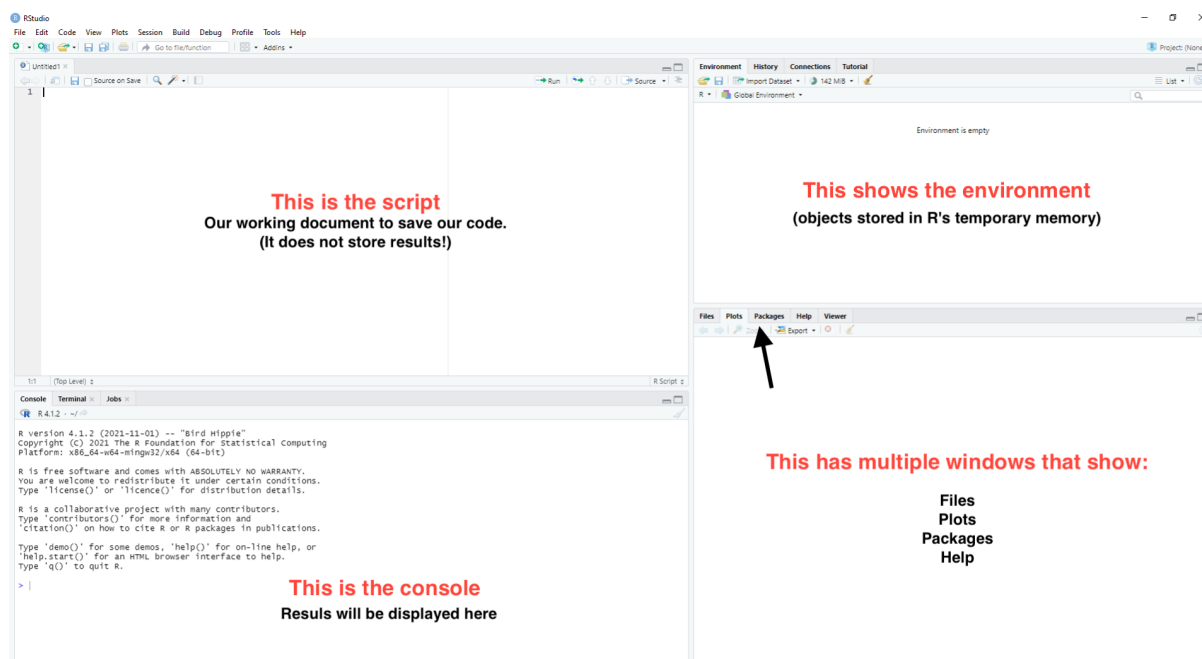
The bottom right corner, contains five tabs with different functionalities of the IDE.

- **Files:** This tab displays the elements on the current working folder. Files in this and in other folders can be deleted, renamed and visualized (with the three dots “...”) in this tab.
- **Plots:** This is the display that shows the graphs produced.
- **Packages:** It shows a list of the currently installed packages. (More about this to come during the week).
- **Help:** In this tab we can see the help resources. We can make general searches or specifically into a topic.
- **Viewer:** for displaying interactive or HTML-based content generated in your R session.

At the top right window, there is a window with multiple tabs. The environment contains all the elements named during the code.

Finally, the top left window is where we type the **script (or code) we are writing and saving for future usages**. If this window is not in the IDE, it means we have to open a new script.

This is done by clicking: File -> New File -> RScript. The script is a text archive that contains ONLY the code that will be processed in the console.



## 4 Before starting

### 4.1 Organizing your work

Everything you hand out in this course is a script that someone else (or you in the future) will open and run. That makes organisation a fundamental part of the work, not an extra.

- Keep one folder per project, with the data and the scripts inside it. When needed, in your script you will point to that folder so the code can still work.
- Name files so that you can recognize them in six months: no spaces, no special characters, no accents. For example `day1_fundamentals.R`, not `script final (2).R`.
- Give your script a header: your name, the date, and one line saying what the script does.
- Write your script in a fixed order: first the packages, then the data, then the analysis. Anyone reading it should be able to run it from top to bottom.
- Annotate as you go, and comment the reason rather than the action. For example:  
`# convert to factor so it can be used in the model is useful, # convert to factor is not.`

These are recommendations from different experienced R users. It is not necessary for coding, but it is always good to have a “good hygiene”.

### 4.2 Good practice advices

- When you start, focus on making the code to work rather than efficient. Efficiency will come along as you learn the program.
- Always code in scripts and not in the console. The console is for testing one line at a time, but all your real work should be in saved scripts you can run again.
- Save your script often.
- Annotate your script. Use `#` before annotating!

- Read the error messages.
- Don't be afraid to try things. Coding is a low-stakes environment where mistakes are easily reversible.

### 4.3 Using AI tools in this course

You almost certainly already use ChatGPT, Copilot or similar tools. Ignoring that would not make you better at R, so let us be explicit about where they help and where they hurt.

What they are good at:

- Writing boilerplate you have written many times before.
- Explaining an error message in plain language.
- Reminding you of the name of a function you have forgotten.

Where they fail, and why it is hard to notice:

- They invent arguments and functions that do not exist, written in perfectly plausible R.
- They use outdated syntax from older versions of a package.
- They produce code that runs without error and answers a different question from the one you asked.

The last one is the dangerous one. Code that fails is a nuisance, code that runs and is wrong is a wrong result in your work. Understanding what AI generated code is doing, requires understanding R yourself. So does troubleshooting a code. Writing code and testing it gives us knowledge about the data we are analyzing. That is useful for making the right decisions about the analysis and visualization. Skipping this step is a very common mistake.

The rule for this course: on the exam you are allowed to use your class notes but not the internet and not AI tools. Anything you cannot do from your own notes and your own understanding is a gap, and you are always welcome to reach out to me or come to the office hours.

## 5 Create an object

To create an object in R, we need to give it a name followed by the assignment operator `<-` and the value. The symbol `=` can also be used for assigning values in most cases but `<-` is the traditional R style and what most R programmers prefer. Once we have assigned the value to a “name”, we can call it again just by its name.

### 5.1 Some rules for creating an object:

- The object can have (almost) any name.
  - Names cannot start with a number.
  - Names cannot contain special characters (e.g. `Apple!`, `Star#`).
- The name should be explicit, but not too long.
- R is case sensitive, so that “data” is different than “Data” and from “DATA”.
- Some names cannot be used as they are the name of specific functions (e.g. `mean`, `sum`, `if`).
- Best to be consistent in the styling of your code.

```
# Object named A will be storing the value 1
A<- 1
# Calling number one by using the object A
A
```

```
## [1] 1
```

NB: The objects we create are going into the environment!

```
# Other ways of naming variables  
var1<- 1  
var.1<- 1  
var_1<- 1
```

## 6 Data type

R can handle different types of data:

- Numeric: Numbers
- Integer: Whole numbers
- Logical: True/ False values
- Character: Text

The data type of an object can be found with the function `class()`.

```
class(A)
```

```
## [1] "numeric"
```

### 6.1 Numeric

The numeric data type represents all real numbers, with or without decimal. For example:

```
height<- 170  
weight<- 65.2
```

```
height
```

```
## [1] 170
```

```
class(height)
```

```
## [1] "numeric"
```

```
weight
```

```
## [1] 65.2
```

```
class(weight)
```

```
## [1] "numeric"
```

### 6.2 Integer

The integer data type represents whole numbers, without decimal points. We use the suffix `L` to tell R explicitly to store a number as an integer. For example:

```
height_integer<- 170L  
class(height_integer)
```

```
## [1] "integer"
```

NB: in practice, most numbers you work with in R are numeric. You will mainly meet integers when you import a dataset and R decides that a column of whole numbers (a year, a count, a code) is an integer.

### 6.3 Logical

The logical data type only have two values TRUE and FALSE. For example:

```
#Write directly the values
L1<- TRUE
L2<- FALSE

#Find if the condition is true or false
L3<- height==weight
L3
```

```
## [1] FALSE
```

```
class(L3)
```

```
## [1] "logical"
```

The logical data can also have a single letter: T for TRUE and F for FALSE.

### 6.4 Character

The `character` data type stores character and string (a chain of characters) values in a variable. In programming `w` is a character and `weight` is a string. We need to use quotation marks to define a character in R. For example:

```
name<- "Paola"
```

```
name
```

```
## [1] "Paola"
```

```
class(name)
```

```
## [1] "character"
```

### 6.5 Factor

A factor stores a categorical variable, that is, a variable that can only take a limited set of values. Those values are called the levels of the factor. Sex, region, income group and education level are all factors.

```
sex <- c("female", "male", "female", "female")
class(sex)
```

```
## [1] "character"
```

```
sex_factor <- as.factor(sex)
class(sex_factor)
```

```
## [1] "factor"
```

```
levels(sex_factor)
```

```
## [1] "female" "male"
```

A factor looks like text when you print it, but R treats it differently: only the defined levels are allowed, and functions that work with categories (tables, plots, statistical models) use those levels. This is why when using such functions, we need to convert categorical variables to factors.

## 6.6 Convert from one type to another

It is possible to convert an object of a certain type to another type using the function `as.numeric()`, `as.integer()`, `as.logical()`, `as.character()` and `as.factor()`. For example:

```
weight <- 65.2
```

```
#Convert to integer
```

```
weight_integer <- as.integer(weight)
```

```
weight_integer
```

```
## [1] 65
```

```
class(weight_integer)
```

```
## [1] "integer"
```

```
# Compare
```

```
weight == weight_integer
```

```
## [1] FALSE
```

```
#Convert to character
```

```
weight_character <- as.character(weight)
```

```
weight_character
```

```
## [1] "65.2"
```

```
class(weight_character)
```

```
## [1] "character"
```

```
# Convert to factor
```

```
BMI <- c("underweight", "normal weight", "overweight", "obesity")
```

```
class(BMI)
```

```
## [1] "character"
```

```
BMI <- as.factor(BMI)
```

```
class(BMI)
```

```
## [1] "factor"
```

Converting is something you will do constantly with real data, because R does not always guess the right type when it imports a file.



## 7 Vector

A vector is a collection of objects of the *same type*. It is created using the `c()` function. The type of vector can be checked with the function `class()`. An important property of the vector is its length, which can be checked with the function `length()`.

```
#Numeric
x<-c(2,4,6,8,10)
x
```

```
## [1]  2  4  6  8 10
```

```
class(x)
```

```
## [1] "numeric"
```

```
length(x)
```

```
## [1] 5
```

```
#character
y<-c(1,2,TRUE,"R")
y
```

```
## [1] "1"    "2"    "TRUE" "R"
```

```
class(y)
```

```
## [1] "character"
```

```
length(y)
```

```
## [1] 4
```

NB: you may occasionally see the function `typeof()`, which reports how R stores the object internally, i.e. what is the object made of inside the container. It sometimes gives a different answer from `class()` ("double" instead of "numeric", for instance), but for most user cases `class()` is sufficient.

### 7.1 Creating vector with :

The operator `:` create a vector of consecutive integer numbers. For example:

```
x<-1:5
x
```

```
## [1] 1 2 3 4 5
```

```
y<--2:2
y
```

```
## [1] -2 -1  0  1  2
```

### 7.2 Creating vector with `seq()`

The function `seq()` allows to create more complex numerical sequences. With `seq()` we can specify the step size or interval. For example:

```
#The argument by defined the width of the interval
x<-seq(0, 20, by=5)
x
```

```
## [1] 0 5 10 15 20
```

```
# Also allows decimal numbers
x<-seq(0,0.9,by=0.1)
x
```

```
## [1] 0.0 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9
```

```
#The argument length.out defined the length of the vector
y<-seq(0, 20, length.out=3)
y
```

```
## [1] 0 10 20
```

### 7.3 Selecting elements of a vector

You can select specific element of a vector, by specifying its position in the vector between squared brackets.

```
x<-seq(0, 20, by=5)
x
```

```
## [1] 0 5 10 15 20
```

```
#Select the first element only
x[1]
```

```
## [1] 0
```

```
#Select the second and fourth elements
x[c(2,4)]
```

```
## [1] 5 15
```

```
#Select all elements, except the first
x[-1]
```

```
## [1] 5 10 15 20
```

```
#Select elements using a logical vector
x[c(T,F,T,F,T)]
```

```
## [1] 0 10 20
```

### 7.4 Modyfing elements of a vector

Select the element to be modified and impose a new value.

```
x
```

```
## [1] 0 5 10 15 20
```

```
#Impose a value
x[2]<-0
```

```
x

## [1]  0  0 10 15 20
#Impose a condition
x[x<10]<-5

x

## [1]  5  5 10 15 20
```

## 7.5 Name the elements

You can attribute a name to each element of a vector.

```
#Directly in the vector
x<-c("A"=1, "B"=2, "C"=3)
x

## A B C
## 1 2 3

#Using the function name()
y<-1:3
names(y)<-c("A", "B", "C")
y
```

```
## A B C
## 1 2 3

#Select element from the position
y[1]

## A
## 1

#Select element from the name
y["A"]

## A
## 1
```

## 8 Basic arithmetic

We can perform basic arithmetic operations on objects and vectors using the standard operators + for addition, - for subtraction, \* for multiplication and / for division.

```
A<-1
B<-2

#Addition
A+B

## [1] 3
```

```
#Subtraction
```

```
A-B
```

```
## [1] -1
```

```
#Multiplication
```

```
A*B
```

```
## [1] 2
```

```
#Division
```

```
A/B
```

```
## [1] 0.5
```

## 9 Functions

Almost everything in R is done through functions. There are some built-in functions in R. For example `sum()`, `mean()`, `round()`, `min()`, `max()`, etc. The input values (inside the parenthesis) are called arguments. Function names are always followed by a parenthesis. A function can have one or multiple arguments and these can be required or optional.

```
# c(), the way we created vectors, is also a function
```

```
x<-c(1,2,4,5,7)
```

```
#Sum of the vector
```

```
sum(x)
```

```
## [1] 19
```

```
#Mean of the vector
```

```
mean(x)
```

```
## [1] 3.8
```

```
#Minimum value of the vector
```

```
min(x)
```

```
## [1] 1
```

```
#Maximum value of the vector
```

```
max(x)
```

```
## [1] 7
```

```
#Round a numeric value, with n digits after the comma
```

```
# Functions can identify arguments by position
```

```
round(3.1415936)
```

```
## [1] 3
```

```
# Or by name
```

```
round(x=3.1415936, digits=3)
```

```
## [1] 3.142
```

## 10 Importing a dataset

There are multiple ways to open a dataset in R, depending on the format of the file: `read.table()` for txt files, `read.csv()` for csv files, `load()` for RData workspace, etc. It is important that you know which type of file you want to import. That is known by looking at the extension (ending) of a file. For example, a file called `file.csv` is a csv file, and file called `file.xlsx` is an Excel file and so on.

To open a dataset, we first need to tell R where to find it! We use the function `setwd()` to set the work directory. This means the way we indicate to R, in which folder in the computer our files are located. You can enter the path manually or using the menu Session > Set Working Directory > Choose Directory.

NB: A good advice is to `setwd()` at the beginning of the script!

```
#setwd("C:/Users/.../R summer school")
```

NB! The path in the working directory is written between quotation marks (“”) because it is a string type!

Then, use one of the different reading function to open the dataset depending on the format of our file. Since the info we are using is a `.csv` file, we will use the function `read.csv()`. That function considers the following arguments:

- The name of the file should be in between quotation marks.
- header: If the first line in the dataset is the name of the variable, header should be `TRUE`.
- skip: Should some lines be skipped. States the row numbers
- sep: What separates the columns (commas, semi-colon, space, etc.)?

```
#Open the data on life expectancy in Denmark  
#First line is the name of the variable  
#Columns seperated by commas
```

```
data<-read.csv("LifeExpectancy_Denmark.csv", header = T, sep=",")
```

```
head(data)
```

```
##   Year LifeExpectancy  
## 1 1800             37.4  
## 2 1801             38.5  
## 3 1802             44.4  
## 4 1803             44.8  
## 5 1804             42.8  
## 6 1805             43.0
```

To know what type of variables we have in the imported data, we can use `class()` as before:

```
class(data$Year)
```

```
## [1] "integer"
```

```
class(data$LifeExpectancy)
```

```
## [1] "numeric"
```

For simplicity, we can create a vector of life expectancy with the year as name. To select a

column in a dataset, we can use `$` between the name of the dataset and the name of the variable. More information on how to handle datasets on the following days.

The first and last few observations of the data can be seen with the functions `head()` and `tail()`.

```
LE<-data$LifeExpectancy
Year<-data$Year
names(LE)<-Year
head(LE)
```

```
## 1800 1801 1802 1803 1804 1805
## 37.4 38.5 44.4 44.8 42.8 43.0
```

```
tail(LE)
```

```
## 2019 2020 2021 2022 2023 2024
## 81.1 81.2 81.1 81.6 81.7 81.9
```

## 11 Basic plot

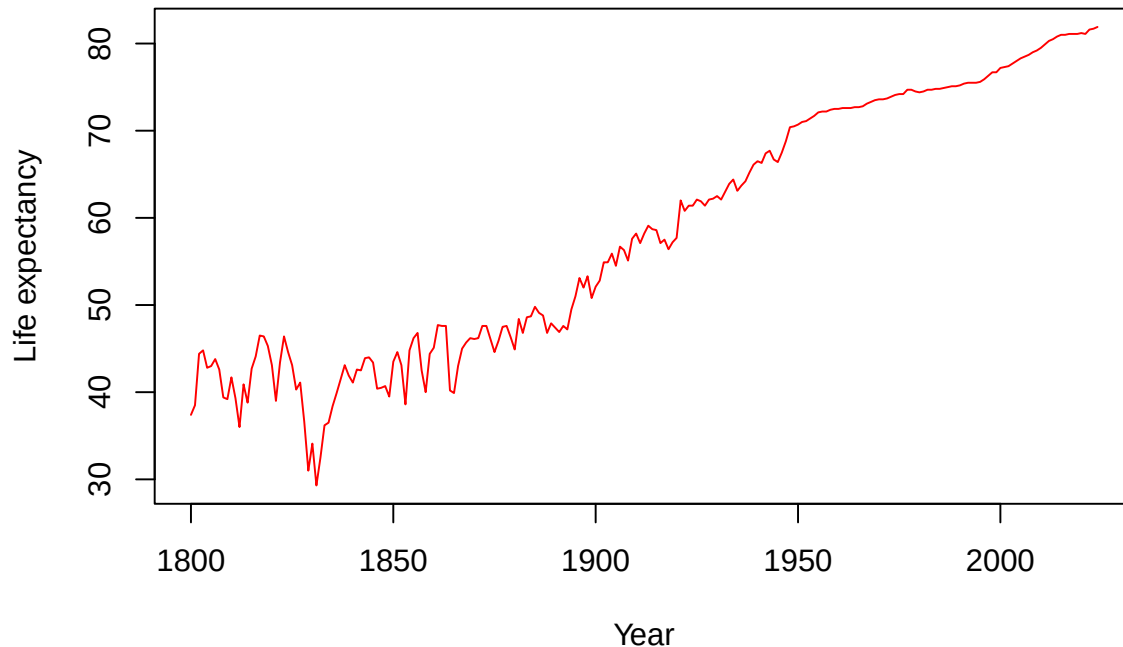
R provides a simple function to plot results `plot()`. This is a first quick plot, made to see what the data look like. In general, base R plots are for looking, not for showing. Visualization has its own session on Day 6, where we will actually discuss more on what it means doing a plot for a report and how to do “pretty” plots.

For using the function `plot()` the arguments are:

- `x`: Variable on the horizontal axis.
- `y`: variable on the vertical axis.
- `type`: Should it be points (“p”, default), lines (“l”) or both (“b”). See manual for more options.
- `col`: color of the points or lines (default:black). Link to color cheat-sheet: <http://www.stat.columbia.edu/~tzheng/files/Rcolor.pdf>
- `pch`: Type of points (1=circle (default), 2=triangle, 3=cross, etc.). Link to cheat-sheet for point type: <http://www.sthda.com/english/wiki/r-plot-pch-symbols-the-different-point-shapes-available-in-r>
- `lty`: Type of lines (1=full, 2=dashed, 3=dotted, etc.). Link to cheat-sheet for linetype: <http://www.sthda.com/english/wiki/line-types-in-r-lty>
- `main`: Title of the plot.
- `xlab`: Name of the x-axis.
- `ylab`: Name of the y-axis.

```
plot(x=Year, y=LE, type="l",
     col="red",
     main="Life expectancy in Denmark between 1800 and 2024",
     xlab="Year",
     ylab="Life expectancy")
```

## Life expectancy in Denmark between 1800 and 2024



## 12 Glossary of functions

The functions and operators introduced today, grouped by what you use them for. Use this as a checklist. If you cannot say what one of these does, or understand it, that is what to review before Day 2.

| Function                                  | What it does                          |
|-------------------------------------------|---------------------------------------|
| <b>Objects and vectors</b>                |                                       |
| <code>&lt;-</code> or <code>=</code>      | Assigns to an object                  |
| <code>c()</code>                          | Combines elements into a vector       |
| <code>:</code>                            | Sequence of consecutive integers      |
| <code>seq()</code>                        | Sequence with a chosen step or length |
| <code>names()</code>                      | Names the elements of a vector        |
| <b>Checking your data</b>                 |                                       |
| <code>class()</code>                      | Data type of an object                |
| <code>length()</code>                     | Number of elements                    |
| <code>levels()</code>                     | Categories of a factor                |
| <code>head()</code> , <code>tail()</code> | First and last observations           |
| <b>Converting types</b>                   |                                       |

| Function                                              | What it does                            |
|-------------------------------------------------------|-----------------------------------------|
| <code>as.numeric()</code> , <code>as.integer()</code> | To numeric, to whole numbers            |
| <code>as.character()</code>                           | To text                                 |
| <code>as.factor()</code>                              | To a categorical variable               |
| <b>Arithmetic with vectors</b>                        |                                         |
| <code>sum()</code> , <code>mean()</code>              | Total, average                          |
| <code>min()</code> , <code>max()</code>               | Smallest, largest                       |
| <code>round()</code>                                  | Rounds to n decimals                    |
| <b>Selecting</b>                                      |                                         |
| <code>[ ]</code>                                      | Elements by position, name or condition |
| <code>\$</code>                                       | A column from a dataset                 |
| <b>Importing data</b>                                 |                                         |
| <code>setwd()</code>                                  | Sets the working folder                 |
| <code>read.csv()</code>                               | Opens a .csv file                       |
| <code>plot()</code>                                   | Draws a basic plot                      |

## 13 Exercises

- Create a vector of odd numbers only between 1 and 10. What is the length of the vector?
- From the vector LE created above, find the life expectancy in Denmark in 1850, 1950 and 2024 and store each of them in a separate object.
- Calculate the mean life expectancy between 1924 and 2024. (Average= $\frac{(A+B)}{2}$ ).
- Create two new vectors: one containing only the life expectancy values in Denmark between 1950 and 2024 and another with the years between 1950 to 2024.
- Plot the life expectancy in Denmark between 1950 and 2024. Design the plot as you wish! Discuss your findings.